

POLITECNICO
MILANO 1863

SCUOLA DI INGEGNERIA INDUSTRIALE
E DELL'INFORMAZIONE

Executive Summary of the Thesis

Towards fast and side-channel resistant implementations of Hamming Quasi-Cyclic: a case study on ARM Cortex-M4 devices

Laurea Magistrale in Computer Engineering - Ingegneria Informatica

Author: Domenico Cacace

Advisor: Prof. Alessandro Barenghi

Co-advisor: Prof. Gerardo Pelosi

Academic year: 2022-2023

---

1. Introduction

The advancements in quantum computing threaten the *status quo* in the cryptography field: a quantum computer with a sufficient amount of qubits is able, thanks to Shor's algorithm, to break most of the current public key cryptosystems in polynomial time.
For this reason in 2016 the National Institute of Standards and Technologies (NIST) launched a competition with the aim of developing and standardizing one or more quantum-resistant cryptosystems. After three rounds CRYSTALS-Kyber, a lattice-based cryptosystem, was selected as standardization candidate; however, in the event of a potentially disruptive attack on lattice-based cryptosystems, NIST decided to further extend the competition selecting four more candidates that rely on different hard problems. Excluding SIKE, the remaining candidates, namely BIKE, HQC and Classic McEliece are all based on the hardness of the syndrome decoding problem.
The object of this work is HQC [6], a code-based cryptosystem whose security relies on the syndrome decoding problem for quasi-cyclic codes. HQC comprises both a Public Key Encryption (PKE) scheme and a Key Encapsulation Mechanism (KEM), obtained by applying the HHK Fujisaki-Okamoto transformation on the former.

Related work Given the sound security record of HQC, much attention has been devoted to its side-channel resistance. Most of the independent analyses focus on extracting information from electromagnetic and power traces, whilst at the end of Round 2 of the NIST contest two timing attack that takes advantage of the non constant-time implementation of the BCH decoder have been shown; these attacks are no longer relevant, as the authors of HQC swapped the BCH code with a Reed-Muller Reed-Solomon concatenated code, implementing a constant-time decoder. Nontheless a new timing attack has been discovered, which exploits a weakness in the *rejection sampling* procedure, employed to generate random sparse polynomials; as a countermeasure, a more care careful implementation of the rejection sampling procedure has been implemented in Round 4, fixing the issue.
To date, no dedicated constant-time implementation of HQC targeting embedded systems has been proposed, despite the fact that side channel attacks are easier to carry on such platforms.

The current state of software implementations of HQC tailored for the ARM Cortex-M4 microcontroller, chosen by the NIST as the reference platform for microcontroller benchmarks, includes only the one in PQM4, a third-party public project that collects the implementations of NIST Round 4 candidates tailored for Cortex-M4. The latest version of HQC in PQM4 is directly derived from the Round 3 reference implementation, with no optimizations: as a consequence, this implementation is not constant-time and vulnerable to the timing attack presented in [4].

Our contribution In this work we present the first fully constant-time software implementation of HQC, together with a comparative analysis of constant-time binary polynomial multiplication techniques, both tailored for ARM Cortex-M4 microprocessors. We provide an experimental validation of the constant-time properties via a battery of statistical t-tests, while varying the CPU frequency to highlight the stability of our solution across different memory access latencies. Our results show that the optimal solution for the target platform is a FFT-based approach, in contrast with classic divide-et-impera techniques (*i.e.* Karatsuba) which provide better performance on x86 architectures.
Moreover, to counter power attacks, we propose a masking scheme that can be applied to the entire cryptosystem, presenting our results on its potential efficacy. Finally, we compare our work with the latest implementations available of BIKE and Classic McEliece, showing the high efficiency of our approaches.

2. Background

In this section we lay the theoretical foundations required to understand our work. To avoid confusion, throughout this work we will use the following notation, unless specified otherwise. Capital bold letters $(\mathbf{A}, \mathbf{B}, \cdots)$ denote generic sets, while lowercase bold letters $(\mathbf{v}, \mathbf{w}, \cdots)$ denote vectors. Binary Galois fields of dimension n are denoted by $\mathbb{F}_{2^n}$ or $GF(2^n)$, while the vector space of dimension n over the field $GF(2)$ is denoted by either $\mathbb{F}_2^n$ or $\mathcal{V}$. The ring of binary polynomials modulo $(x^n - 1)$ is denoted by $\mathcal{R}$; the value of n is chosen in such a way that the algebra in the ring $\mathcal{R}$ is isomorphic to the algebra of binary $n \times n$ circulant matrices.

2.1. Polynomial multiplication

The most expensive operation in terms of execution time in our cryptosystem is, by far, the polynomial multiplication: it is thus essential to choose a fast algorithm in order to increase the overall performance.

2.1.1 Karatsuba Multiplication

One of the most known fast multiplication algorithms is Karatsuba. This is a divide-et-impera algorithm, which splits the n-degree input polynomials into smaller m-degree ones:

$$a(x) = a_0(x) + x^m \cdot a_1(x)$$

With the operands in this form, it is possible to rewrite the product of two polynomials as

$$c(x) = z_2(x) \cdot x^{2m} + z_1(x) \cdot x^m + z_0(x)$$

with

$$\begin{aligned} z_2(x) &= a_1(x)b_1(x) \\ z_1(x) &= a_1(x)b_0(x) + a_0(x)b_1(x) \\ z_0(x) &= a_0(x)b_0(x) \end{aligned}$$

Karatsuba observed that, at the cost of a few more additions, it is possible to calculate the product using only three multiplications by rewriting z_1 as

$$(a_0(x) + a_1(x))(b_0(x) + b_1(x)) - z_0(x) - z_2(x)$$

This trick allows to calculate the product, under the assumption that $m = \lceil n/2 \rceil$, with $\mathcal{O}(n^{\log_2 3})$ operations.

2.1.2 Frobenius Additive FFT

Instead of multiplying two polynomials (of degree less than n) directly, we can:

- Sample the polynomials at $2n + 1$ distinct points $(a(i), b(i))$
- Perform the pointwise multiplication of the samples $(c(i) = a(i)b(i))$
- Interpolate the points to get the product of the original polynomials

This approach may appear slower than the schoolbook method, as the sampling requires $\mathcal{O}(n^2)$ operations, the pointwise product $\mathcal{O}(n)$

and the interpolation up to $\mathcal{O}(n^3)$ with the Gauss elimination method. However, by recursively splitting the operands and by picking a set of *special* point, such as the n-th roots of unity in $\mathbb{C}$, the sampling and interpolation steps can be performed with the Discrete Fourier Transform and its inverse: algorithms that perform this procedure are commonly called Fast Fourier Transforms (FFTs), and can achieve a complexity of $\mathcal{O}(n \log n)$ operations.
Fields of characteristic two usually do not admit non-trivial roots of unity; nontheless, we can apply the same principle by picking as evaluation points the powers of a primitive element of the field: these algorithms are called Additive FFTs. An example of Additive FFT is the Lin-Chung-Han (LCH) FFT which, for polynomials of degree $n = 2^m$, calculates the product in $\mathcal{O}(n \log n)$.

New polynomial basis In [3] Gao and Mateer use a special basis to represent the polynomials for their FFT. For polynomials of degree $n = 2^m$, the Cantor basis $\mathbf{v} = (v_1, \cdots, v_m)$ of $\mathbb{F}_2^m$ follows the structure:

$$v_1 = 1, \qquad v_i^2 + v_i = v_{i-1} \text{ for } i > 1$$

With respect to this basis we can define a sequence of *nested* subspaces:

$$\mathbf{W}_i := span\{v_0, \cdots, v_{i-1}\}, \quad \mathbf{W}_0 \subset \cdots \subset \mathbb{F}_2^m$$

and, for each subspace, its vanishing polynomial:

$$s_i(x) = \prod_{a \in \mathbf{W}_i} (x - a)$$

For the LCH FFT we first need to represent the polynomials in a different basis, built onto the vanishing polynomials; this basis $(X_1, \cdots, X_{n-1})$ is called *novelpoly basis*, and is defined as:

$$X_k(x) = \prod (s_i(x))^{b_i} \quad \text{with } k = \sum_{i=0}^{\log_2 n} b_i 2^i$$

The butterflies As with other FFT, in the LCH FFT we recursively split the input polynomial into two halves; we also split the set of evaulation points $\mathbf{W}_k$ into two disjoint sets, $\mathbf{W}_{k-1}$ and $\mathbf{W}_k \setminus \mathbf{W_{k-1}} = \{x + v_k | x \in \mathbf{W}_{k-1}\}$. Because of the linearity of the vanishing polynomial $(s_i(x+y) = s_i(x) + s_i(y))$, the evaluations of the input polynomial at $\mathbf{W}_k \setminus \mathbf{W_{k-1}}$ can be quickly computed from the evaluations at $\mathbf{W}_{k-1}$ with the *butterfly* operation, shown at lines 6 and 7 in Algorithm 1

Algorithm 1: Lin-Chung-Han FFT

Input: $f(x)$, in novelpoly basis,
$\alpha \in \mathbb{F}_{2^m}$, a generic field element
Output: $\{f(x) | x \in \mathbf{W}_m\}$

```
1 Function FFT(f(x), α)
2     if deg(f(x)) then
3         return [g_0];
4     k ← max i s.t. 2^i ≤ deg(f(x));
5     g(X) ← p_0(X) + X_{2^k} · p_1(X);
6     h_0(X) ← p_0(X) + s_k(α) · p_1(X);
7     h_1(X) ← h_0(X) + s_k(v_k) · p_1(X);
8     return FFT(h_0(X), α),
          FFT(h_1(X), v_k + α);
```

Notice that the butterfly carries out two field additions but only one field multiplication, because $s_k(v_k) = 1$ in Cantor basis ([1], Appendix A). The output of this algorithm is the set of evaluation points of the input polynomial; these can be multiplied with the evaluations of a second polynomial and, by performing the FFT and basis conversion backwards, we can obtain the product of the two.

Frobenius partition In [7] the authors showed that, by applying the Frobenius map $(\phi : x \mapsto x^2)$, it is possible to derive more values of a polynomial from less evaluation points: given a polynomial $f(x)$ of degree at most n, we can design a set of points $\mathbf{S} \subset \mathbb{F}_{2^m}$ for m power of 2 such that we can derive n evaluations of f from $\mathbf{S}$. In [5] the authors suggest to pick $\mathbf{S}$ as:

$$\mathbf{S} := v_{l+m/2} + \mathbf{W}_l = \{v_{l+m/2} + x | x \in \mathbf{W}_l\} \quad (1)$$

with $l = \log_2 n - \log_2 m$ and size $n' = |\mathbf{S}| = |\mathbf{W}_l| = 2^l = n/m$. As explained by the authors, this construction allows to reach n points in $\mathbb{F}_{2^m}$ by applying the Frobenius map, which also ensures that the evaluation function

$$E_{\mathbf{S}} : f(x) \mapsto f(\mathbf{S}) = \{f(v_{l+m/2+w}) | w \in \mathbf{W}_l\}$$

will be invertible.

2.2. Side-channel attacks

Even though a cryptographic primitive may be secure from a theoretical point of view, its implementation may not. It is possible that a device exposes some *side channels*, observable aspects of a system that may reveal information about the internal state of the system; these side channels, if the device runs a cryptosystem, be exploited to *leak* secret information, such as a message or the keys employed to decrypt it.
Two of the most common side-channel attacks, especially on embedded devices, are:

- **Timing attacks**: the execution time of an operation may depend on the values of its inputs, allowing an attacker to retrieve information by measuring the execution time
- **Power attacks**: the amount of power required by the CPU varies based on the workload, which directly depends on the operations being run and their inputs

The presence of a side channel does not necessarily mean that an attack is possible, as the leaked information may not be sufficient to succesfully exfiltrate information. If this is not the case, there are various countermeasures to mitigate the risk of a successful attack, such as:

- Constant-time algorithms: running an operation takes always the same amount of time, regardless of the input (*e.g.* substitute branching instructions depending on the inputs)
- Masking: randomize the inputs of an algorithm without affecting the outputs; this can be achieved by splitting sensitive data x into $d+1$ *shares*, such that $x = x_0 \oplus \cdots \oplus x_d$, making the sensitive value indistinguishable from random noise.

2.3. Coding theory

To better understand how HQC works, we now give the basic concepts of coding theory.

Definition 2.1 (Circulant matrix)**.** Let $\mathbf{v} = (v_0, \cdots, v_{n-1}) \in \mathcal{V}$. The circulant matrix induced by $\mathbf{v}$ is defined as a matrix whose i-th row is the vector $\mathbf{v}$ shifted by i positions to the right:

$$\mathbf{circ}(\mathbf{v}) = \begin{pmatrix} v_0 & v_1 & \cdots & v_{n-1} \\ v_{n-1} & v_0 & \cdots & v_{n-2} \\ \vdots & \vdots & \ddots & \vdots \\ v_1 & v_2 & \cdots & v_0 \end{pmatrix} \in \mathbb{F}_2^{n \times n}$$

The algebra of $n \times n$ circulant matrices over $\mathbb{F}_2$ is isomorphic to the algebra of polynomials in the ring $\mathcal{R}$

Definition 2.2 (Linear Codes)**.** A linear code $\mathcal{C}$ of lenght n and dimension k (denoted as $[n, k]$) is a subspace of $\mathcal{V}$ of dimension k; the elements of $\mathcal{C}$ are called *codewords*.
Due to the linearity of $\mathcal{V}$, any linear combination of two codewords is also a codeword.

Definition 2.3 (Generator Matrix)**.** Let $\mathcal{C} = [n, k]$ be a linear code. The matrix $\mathbf{G} \in \mathbb{F}_2^{k \times n}$ is called the generator matrix of $\mathcal{C}$ if

$$\mathcal{C} = \{\mathbf{mG}, \text{ for } \mathbf{m} \in \mathbb{F}_2^k\}$$

Definition 2.4 (Parity-Check Matrix)**.** Let $\mathcal{C} = [n, k]$ be a linear code. The matrix $\mathbf{H} \in \mathbb{F}_2^{(n-k) \times n}$ is called the parity-check matrix of $\mathcal{C}$ if

$$\mathcal{C} = \{\mathbf{v} \in \mathbb{F}_2^n \text{ such that } \mathbf{Hv}^\top = 0\}$$

In other words, $\mathbf{H}$ is the generator matrix of the dual code $\mathcal{C}^\perp$.

Definition 2.5 (Syndrome)**.** Let $\mathcal{C} = [n, k]$ be a code, $\mathbf{H}$ its parity matrix and $\mathbf{v} \in \mathbb{F}_2^n$ a word. The syndrome of $\mathbf{v}$ is defined as

$$\mathbf{S}(\mathbf{v}) = \mathbf{Hv}^\top$$

From this definition, a vector $\mathbf{v}$ is a codeword if and only if its syndrome is the zero vector.

Definition 2.6 (Quasi-Cyclic Codes)**.** Let $\mathbf{c} = (\mathbf{c}_0, \cdots, \mathbf{c}_{s-1}) \in \mathbb{F}_2^{sn}$ be a vector, where we consider each $\mathbf{c}_i$ as a vector in $\mathbb{F}_2^n$. A linear code $\mathcal{C} = [sn, k]$ is Quasi-Cyclic of index s if, for any $\mathbf{c} \in \mathcal{C}$, the vector obtained by applying a circular shift to every block $\mathbf{c}_i$ simultaneously is also a codeword.

2.3.1 Hard problems for cryptography

The problems we are interested in are all variants of the *decoding problem*, which requires to find the closest codeword to a given word with respect to a certain metric; in our case we consider as metric the Hamming weight, denoted as $\mathtt{wt}(\mathbf{v})$, which is the amount of non-zero coordinates of the vector $\mathbf{v}$. In the case of linear codes we have shown that the decoding problem stays the same whether one is given the syndrome of the received vector or

the vector itself; for this reason, we will only consider the *Syndrome Decoding* (SD) problem.

Definition 2.7 (SD Distribution)**.** Given $n, k, w \in \mathbb{N}$, the distribution $\mathsf{SD}(n, k, w)$ randomly samples a parity-check matrix $\mathbf{H} \xleftarrow{\$} \mathbb{F}_2^{(n-k)\times n}$, a vector $\mathbf{x} \xleftarrow{\$} \mathbb{F}_2^n$ such that $\mathtt{wt}(\mathbf{x}) = w$ and returns $(\mathbf{H}, \mathbf{S}(\mathbf{x}))$.

Definition 2.8 (SD Problem)**.** Given $(\mathbf{H}, \mathbf{y}^\top) \in \mathbb{F}_2^{(n-k)\times n} \times \mathbb{F}_2^{(n-k)}$ from the $\mathsf{SD}(n, k, w)$ distribution, the SD problem requires to find a vector $\mathbf{x} \in \mathbb{F}_2^n$ such that $\mathbf{S}(\mathbf{x}) = \mathbf{y}^\top$ and $\mathtt{wt}(\mathbf{x}) = w$.

This problem has been proven to be NP-complete.

2.4. Hamming Quasi-Cyclic

Hamming Quasi-Cyclic is a code-based cryptosystem, that includes both a public-key encryption scheme and a key encapsulation mechanism. The KEM is built by applying the HHK transformation on the PKE, hence we will focus on the latter. HQC employs two different codes:

- A public decodable code $\mathcal{C}$ generated by $\mathbf{G}$ which can correct up to δ errors efficiently with $\mathcal{C}.\mathsf{Decode}()$.
- A random code with parity-check matrix $\mathbf{H} = [\mathbf{I}_n, \mathbf{circ}(\mathbf{h})]$

The idea at the core of the PKE is to encode the message and add random errors $(\mathbf{r_1}, \mathbf{r_2}, \mathbf{e})$, sparse polynomials with a certain Hamming weight that can be removed only by knowing the secret key: in this way the code $\mathcal{C}$ can decode the ciphertext only if these errors are removed. The authors showed that breaking the PKE scheme from a theoretical point of view is equivalent to solving the decisional version of the SD problem for quasi-cyclic codes, thus proving that the PKE is IND-CPA secure.

Parameters HQC has three different parameters sets for category 1, 3 and 5 respectively; the parameters are summarized in Table 1.

Table 1: Parameters of HQC

Instance	$\mathbf{n_1}$	$\mathbf{n_2}$	$\mathbf{n}$	ω	$\omega_\mathbf{r} = \omega_\mathbf{e}$
`hqc-128`	46	384	17669	66	75
`hqc-192`	56	640	35851	100	114
`hqc-256`	90	640	57637	131	149

Algorithm 2: HQC PKE

Data: $\mathcal{R}_\alpha := \{x \in \mathcal{R} | \mathtt{wt}(x) = \alpha\}$,
$\xleftarrow{\$}$: uniform sampling from r.h.s

Function `KEYGEN()`
- $h \xleftarrow{\$} \mathcal{R}$;
- $(x, y) \xleftarrow{\$} \mathcal{R}_\omega \times \mathcal{R}_\omega$;
- $s \leftarrow x + hy$;
- $\mathcal{K}_{pri} \leftarrow (x, y)$;
- $\mathcal{K}_{pub} \leftarrow (h, s)$;
- **return** $(\mathcal{K}_{pri}, \mathcal{K}_{pub})$

Function `ENCRYPT`$(\mathcal{K}_{pub}, m)$
- $e \xleftarrow{\$} \mathcal{R}_{\omega_e}$;
- $(r_1, r_2) \xleftarrow{\$} \mathcal{R}_{\omega_r} \times \mathcal{R}_{\omega_r}$;
- $u \leftarrow r_1 + hr_2$;
- $v \leftarrow m\mathbf{G} + sr_2 + e$;
- **return** $c = (u, v)$

Function `DECRYPT`$(\mathcal{K}_{pri}, c)$
- **return** $\mathcal{C}.\mathit{Decode}(v - uy)$

3. Implementation

At the time of writing there are no publicly available implementations of HQC specifically targeting microcontrollers. In this section we describe our implementation of HQC for Cortex-M4 microcontrollers, showing the implementative detail of various multiplication algorithms and the details of our masking scheme. Due to its rather large memory footprint, our work focuses on the `hqc-128` version.

3.1. Polynomial multiplication

The polynomials $v \in \mathcal{R}$ can be represented in tow possible forms:

- sparse, as a vector of ω coefficients, $\omega = \mathtt{wt}(v)$, with each entry corresponding to the degree of a non-zero coefficient
- dense, as a vector of size $\lceil \frac{n}{\mathtt{ws}} \rceil$, with `ws` size of a memory word.

One of the operands in the multiplications in HQC has always a very low Hamming weight (75) compared to the maximum degree (17669): can we exploit this property to get fast polynomial multiplications?

Shift&Add The Round 3 reference implementation of HQC implements polynomial multiplication with the *shift and add* (S&A) method: multiplication in $\mathcal{R}$ of a polynomial by

a monomial of degree i can be achieved with a circular shift to the left of i positions ($\lll_n i$) on the polynomial; by fixing the dense input and repeating this procedure for each non-null coefficient of the sparse one and by summing up the partial results we get the product of the two inputs. The low weight of the sparse polynomial causes this approach to be extremely fast; however, this performance is counterbalanced by the non-constant time execution, which is a major drawback.

Constant-Time S&A We would like to exploit the low weight of the sparse operand by implementing a constant-time version of S&A; such an algorithm is implemented in LEDAcrypt and it is based on *shifts with conditional commitment*: we calculate the shifts for each bit β_i required to represent the degree of the monomial, but we sum it to the final result only if that bit is 1.

Algorithm 3: Constant-time S&A

$c(x) \leftarrow b(x) \lll_n (a \mod \texttt{ws})$;
for $i \leftarrow \log_2(\texttt{ws})$ **to** $(m-1)$ **do**
  $\texttt{tmp}(x) \leftarrow b(x) \lll_n i$;
  $c(x) \leftarrow c(x) + \texttt{tmp}$;
  $\texttt{cond} \leftarrow (\beta_i = 1)$;
  $\texttt{mask} \leftarrow 0 - (\texttt{WORD})(\texttt{cond})$;
  $c(x) \leftarrow c(x) + ((c(x) \wedge \texttt{mask}) \vee (\texttt{tmp}(x) \wedge \texttt{mask}))$;
return $c(x)$;

Frobenius FFT Our implementation of the FAFFT follows the same principles of [2]. Our target is a 32-bit CPU, hence we pick $m = 32$; moreover, since the maximum degree of the operands is 17669, we choose n to be the greatest power of 2 greater than $2 \cdot 17669$, hence $n = 65536$. Recalling Equation 1, the optimal Frobenius partition is $\mathbf{S} = v_{27} + \mathbf{W}_{11}$. Since the degree of the operands is fixed, we can perform a variety of optimizations:

- Basis conversion: the recursive operations can be unrolled, generating a circuit with only `XOR` operations where we can discard inputs that we are sure to be zero due to the padding.
- Pointwise multiplication: we can perform the $n/m = 2048$ multiplications in $\mathbb{F}_{2^{32}}$ by applying Karatsuba, so that the results can nicely fit into the general purpose registers.
- Encoding: the first $\log_2 m$ layers of the butterfly step can be rewritten as a vector-matrix product which, similarly to the basis conversion, can be carried out with a binary circuit

3.2. Masking

A peculiarity of HQC is that, if we consider the keys as constants, the whole scheme is linear in all of it operations, making its primitives easy to mask. By denoting with $[\![x]\!]$ the splitting of x into d shares and with $(x + \sum \texttt{rnd}_i)$ the masking of x with $d-1$ random values, we can summarize the masked implementation of the PKE as in Algorithm 4

Algorithm 4: HQC PKE, masked

Function KEYGEN()
  $h \xleftarrow{\$} \mathcal{R}$;
  $(x, y) \xleftarrow{\$} \mathcal{R}_\omega \times \mathcal{R}_\omega$;
  $[\![y]\!] \leftarrow (y + \sum \texttt{rnd}_i)$;
  $[\![s]\!] \leftarrow h[\![y]\!]$;
  $[\![s]\!] \leftarrow x + [\![s]\!]$;
  $\mathcal{K}_{pri} \leftarrow (x, y)$;
  $\mathcal{K}_{pub} \leftarrow (\text{COLLAPSE}([\![h]\!]), s)$;
  return $(\mathcal{K}_{pri}, \mathcal{K}_{pub})$
Function ENCRYPT($\mathcal{K}_{pub}, m$)
  $e \xleftarrow{\$} \mathcal{R}_{\omega_e}$;
  $(r_1, r_2) \xleftarrow{\$} \mathcal{R}_{\omega_r} \times \mathcal{R}_{\omega_r}$;
  $[\![r_1]\!] \leftarrow (r_1 + \sum \texttt{rnd}_i)$;
  $[\![r_2]\!] \leftarrow (r_2 + \sum \texttt{rnd}_i)$;
  $[\![u]\!] \leftarrow [\![r_1]\!] + h[\![r_2]\!]$;
  $[\![m]\!] \leftarrow (m + \sum \texttt{rnd}_i)$;
  $[\![v]\!] \leftarrow [\![m]\!]\mathbf{G} + s[\![r_2]\!] + e$;
  return $(\text{COLLAPSE}([\![u]\!]), \text{COLLAPSE}([\![v]\!]))$
Function DECRYPT($\mathcal{K}_{pri}, c$)
  $[\![y]\!] \leftarrow (y + \sum \texttt{rnd}_i)$;
  $[\![uy]\!] \leftarrow u[\![y]\!]$;
  $[\![\bar{m}]\!] \xleftarrow{\$} (\{0,1\}^k)^d$;
  $[\![\bar{c}]\!] \leftarrow [\![\bar{m}]\!]\mathbf{G}$;
  $a \leftarrow \text{COLLAPSE}((v, [\![\bar{c}]\!]))$;
  $m \leftarrow \text{DECODE}(\text{COLLAPSE}((a, [\![uy]\!])))$;
  return $\text{COLLAPSE}((m, [\![\bar{m}]\!]))$;

In the keygen and encryption algorithms we simply mask the secret values, summing them in

order to avoid leakage; in the decryption we exploit the linearity of the correctable code: after masking y and multiplying by u, we generate a random codeword $[\![\bar{m}]\!]$ and encode it into $[\![\bar{c}]\!]$ to protect $v - uy$ during the decoding. After the decoding we can collapse $[\![\bar{m}]\!]$ and the decoded message: since the code is linear, the final result is the original message, as if we had performed the decoding on $v - uy$ directly.

4. Experimental results

We choose the STM32L476RG Nucleo board as our benchmarking platform, featuring 96KB of RAM, 1MB of Flash memory, a 32-bit ARM Cortex M4 which can be clocked up to 80MHz and a true random number generator. We use `libopencm3` 1.0.0 as firmware library and compile the project with `gcc-arm-none-eabi` 7.2.1 using the `-O3` optimization flag. The board is clocked at a range of frequencies between 16 and 80 MHz to evaluate the impact of the memory wait states. We run the same experiments on a x86 machine, equipped with an AMD Ryzen 25000U, 8GB of DDR4 DRAM and running ArchLinux 6.2; the executables are compiled with `gcc` 12.2.1 using the `-O3` optimization flag and, for AVX implementations (marked with a '†'), the `-march=native` flag.

Welch's t-test To evaluate the existence of an eventual side-channel vulnerability we use the Test Vector Leakage Assessment (TVLA), based on Welch's t-test. The aim of the test is to provide a quantitative measure of how much the means of two populations differ; the t-test statistic is computed as follows:

$$t = \frac{\mu_0 - \mu_1}{\sqrt{\frac{s_0^2}{n_0} + \frac{s_1^2}{n_1}}} \tag{2}$$

where μ, s^2 and n are the mean, variance and number of samples of the two populations, respectively. Small t values give evidence to accept the null hypothesis that the two populations have the same mean; usually, for the sake of simplicity, a threshold of $|t| > 4.5$ is defined to reject the null hypothesis with a confidence level of $99,999\%$.

4.1. Execution time

To measure the execution times on ARM we use the Data Watchpoint and Trace Cycle Count Register (`DWT_CYCCNT`), a 32-bit register that counts the number of clock cycles elapsed since its last reset with a resolution of one clock cycle; to avoid overflows, we reset the register after all the measurements of a single round. As for the x86 machine, we use the `rdtsc` instruction, that returns the current value of the processor's time stamp counter; to get accurate measurements, we lock the frequency of all cores to 2.0GHz using `cpupower` and disable the Turbo Boost feature. In both cases, in order to reduce uncertainty, we run the experiments 1000 times and average the results.
As already mentioned, the multiplication operation is the most time-consuming operation for all primitives, hence the choice of the multiplication algorithm is crucial to achieve good performance. The execution times of the various multiplication algorithms are shown in Figure 1. As expected, the fastest multiplication algorithm on ARM (after the Shift&Add) is the one using the Frobenius FFT, followed by the constant-time version of Shift&Add. In the graph we can also see how the execution time for the Frobenius FFT is the only one that is significantly impacted by the amount of memory wait states, as it relies on precomputed tables, requiring a higher amount of memory accesses and thus up to 92% more clock cycles.
Things change dramatically on x86: on this architecture we can take advantage of AVX instructions, which allow us to carry out operations with 256-bit vectors in a single clock cycle: this benefits the Karatsuba algorithm, which is the fastest one by a large margin, while the Frobenius FFT becomes the slowest one.

4.2. Constant-time execution

As stated by the NIST, the constant-time execution is an important property for a standardization candidate, as it increases its security against side-channel attacks: it is hence important to assess the presence of timing leaks in our multiplication implementations before using them in the PKE and KEM schemes. As already mentioned we employ the TVLA and take as populations the execution times of the algorithm under test, running it with variable and fixed inputs; both populations are sampled 1000 times and, for ARM, the frequency is set to 80MHz. The results of the tests are reported in Table 2.

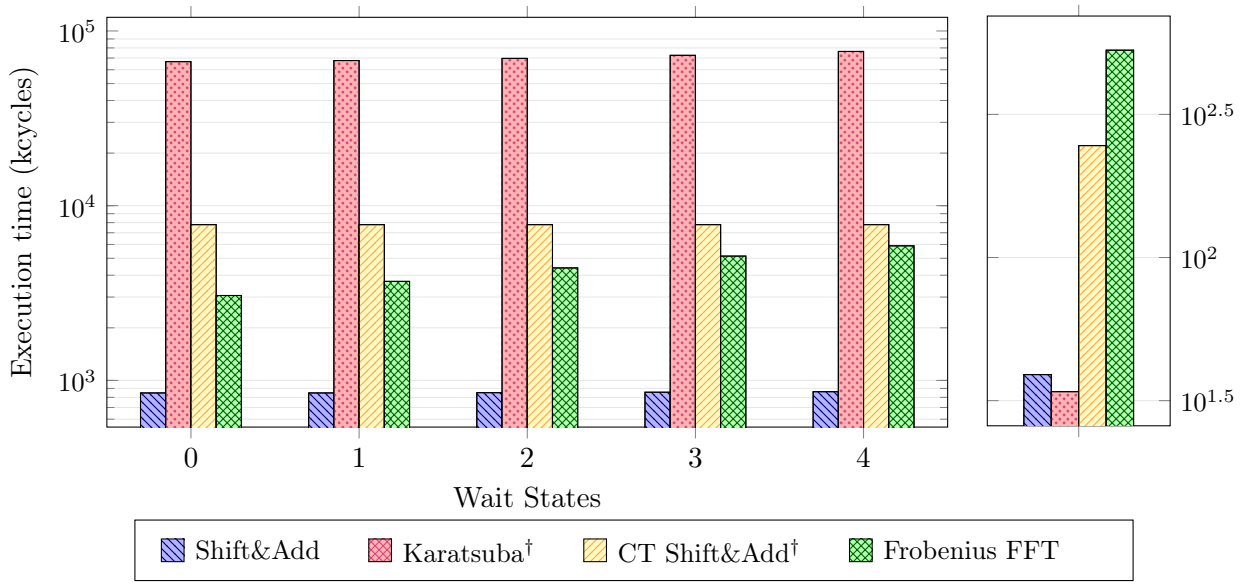


Figure 1: Multiplication execution times on ARM (left) and x86 (right) in kcycles

Table 2: t-values for the multiplication algorithms

	S&A	Karatsuba	CT S&A	FAFFT
ARM	55.748	0.000	0.000	0.000
x86	20.936	3.644	3.371	1.232

As expected, the schoolbook Shift&Add does not run in constant time while the other algorithms do, as they perform always the same operations regardless of the inputs; the constant-time behavior of a part of the system however does not guarantee that the whole system runs in constant time. In [4], the authors show that the algorithm employed in HQC and BIKE to sample polynomials with a fixed Hamming weight is not constant-time and can be exploited to fully recover the secret key; this vulnerability was discovered on the Round 3 version, and it has been fixed in the subsequent round. We therefore run the TVLA on the PKE and KEM schemes with the same settings as before, and find the results reported in Table 3.
We can see that, even with a constant-time multiplication algorithm, the Round 3 version is not constant-time, while the Round 4 version runs in constant time with all the multiplication algorithms.

4.3. Power measurements

Our power measuring setup employs the PicoScope 5244D oscilloscope sampling at 500MS/s, two Agilent INA-10386 amplifiers in a cascade configuration achieving a 52dB gain and a H-probe, placed on top of the STM32L476RG package. The EM traces are recorded and analyzed on a computer equipped with an AMD A8-6600K CPU, 16GB of DDR3 DRAM running Debian 10. To ensure that the encryption/decryption process runs correctly, we generate the inputs on the same machine, send them to the target via the USART1 interface and compare the outputs with the expected ones; the executables to generate the inputs and verify the outputs are compiled with `gcc` (9.3.0) on the same machine, using the `-O3` optimization flag. Due to the significant amount of time required to acquire the EM traces, the populations of our TVLA contain 1000 traces each. Our tests results show no significant leakage, as shown in Figure 2.

Figure 2: t-value statistics on the electromagnetic traces (decryption)

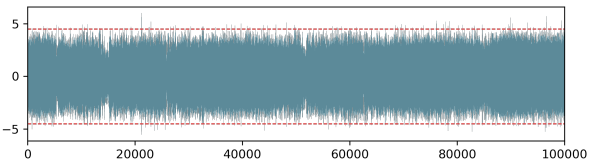


Table 3: t-values for the PKE and KEM algorithms

Round	Mul	CT?	Encrypt		Decrypt		Encaps		Decaps	
			x86	ARM	x86	ARM	x86	ARM	x86	ARM
3	Shift&Add	○	10.65	29.19	5.59	39.23	58.42	28.65	20.71	18.44
	CT Shift&Add†	•	9.38	23.12	7.59	49.56	6.75	14.29	5.81	23.70
4	Karatsuba†	•	1.27	1.04	0.97	0.30	3.77	4.01	4.56	0.26
	CT Shift&Add†	•	0.29	0.07	1.51	1.64	1.58	2.17	1.04	0.22
	Frobenius FFT	•	3.36	0.56	1.70	1.99	4.31	4.25	2.41	0.53

However, similar results are achieved as well on the non-masked version of the algorithms: since other works have proven the existence of a power side-channel attack on the RMRS decoder, we conclude that further measurements are required to assess the security of the masking scheme.

4.4. Comparison with other schemes

Similarly to the NIST Round 3 report, we can compare the performance of our implementation with the other Round 4 candidates; it is interesting to note that all three candidates are code-based cryptosystems and that they all have a Cortex-M4 implementation. In the graphs below we can see the different amount of time required by our implementations, BIKE and McEliece to perform the different operations; for consistency, we consider only the optimized implementations for Cortex M4 with level 1 parameters. For HQC, we consider the results obtained with zero memory wait states, as the authors of the other implementations did.

We can see how the McEliece cryptosystem is the fastest one in terms of encapsulation and decapsulation, but requires $2 \cdot 10^{12}$ cycles to generate the keypair. With respect to BIKE, the most promising candidate in this round, we can see how HQC outperforms it in terms of key generation and decapsulation, not only in the fast implementation but also in the masked one.

In terms of size, we can see how McEliece is prohibitively large in terms of keysize (over 250KB for the public key only). With respect to BIKE, we can see that HQC ciphertext and public keys are larger by a factor of 2.9 and 1.5 respectively.

5. Conclusions

In this thesis we provided the first constant-time implementation of HQC and the first masked implementation of HQC's public key encryption scheme.

To do so, we evaluated different multiplication methods in terms of both performance and constant-time behavior, crowning the Frobenius Additive FFT as the best one on the ARM Cortex-M4 and the Karatsuba algorithm on x86 architectures; moreover, we implemented a masking scheme for the key generation, encryption and decryption primitives of HQC's public key encryption scheme.

We provided a comparative analysis of the Round 4 NIST PQC candidates, showing how both our constant-time and masked implementations of HQC offer a good trade-off in terms of memory consumption and execution time, being faster than the (unmasked) BIKE implementation and thus making them suitable for standardization.

As future work, it could be possible to run more accurate tests in order to evaluate the efficacy of our masking scheme against power attacks. Since our masking scheme targets the PKE only, it could also be possible to extend the masking scheme to the SHAKE functions and make the whole KEM secure against power attacks.

References

[1] Ming-Shing Chen, Chen-Mou Cheng, Po-Chun Kuo, Wen-Ding Li, and Bo-Yin Yang. Faster multiplication for long binary polynomials. *CoRR*, abs/1708.09746, 2017.

[2] Ming-Shing Chen, Tung Chou, and Markus Krausz. Optimizing BIKE for the intel haswell and ARM cortex-m4. *IACR Trans. Cryptogr. Hardw. Embed. Syst.*, 2021(3):97–124, 2021.

[3] Shuhong Gao and Todd D. Mateer. Additive fast Fourier transforms over finite fields.

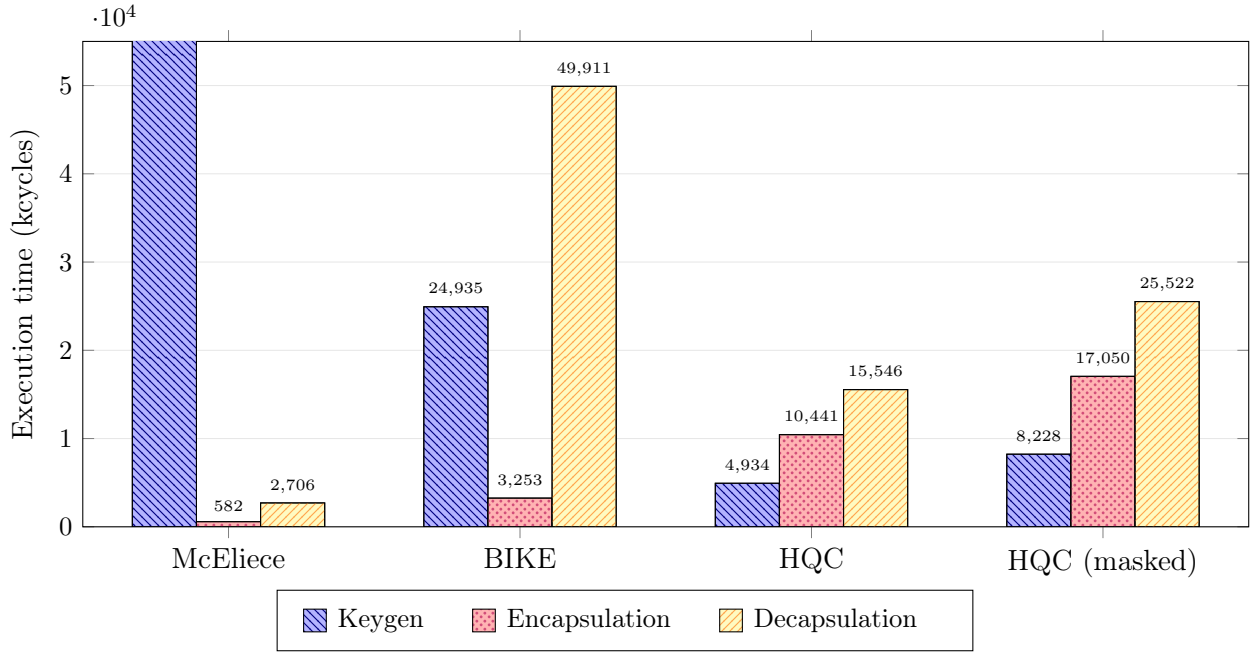


Figure 3: Comparison of execution time for the McEliece, BIKE, and HQC KEMs

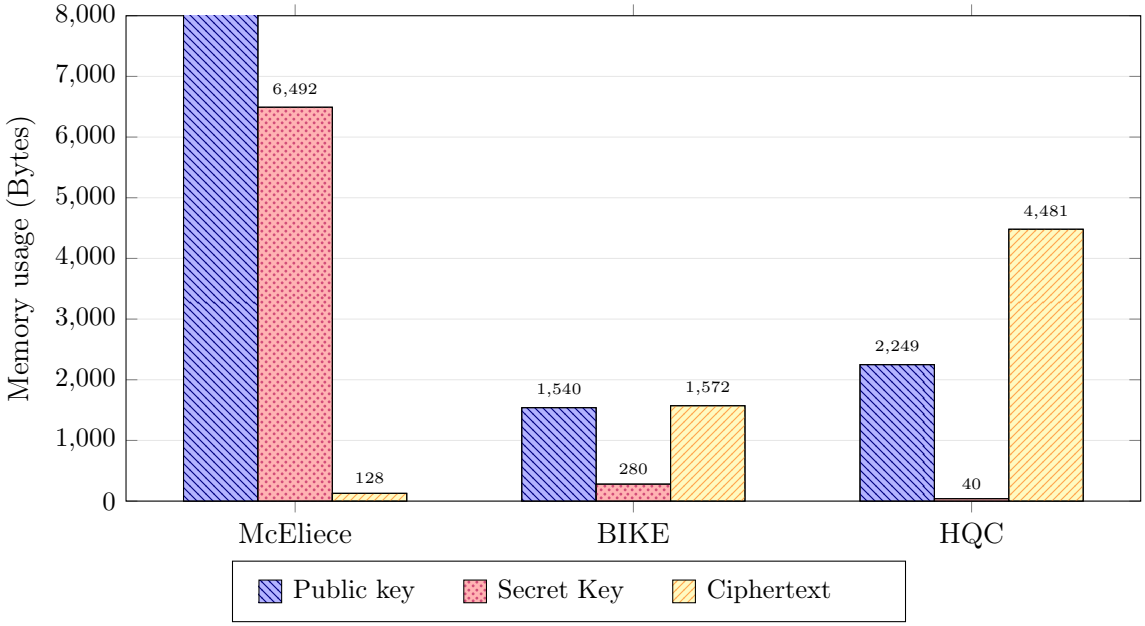


Figure 4: Memory usage comparison of McEliece, BIKE and HQC

IEEE Trans. Inf. Theory, 56(12):6265–6272, 2010.

[4] Qian Guo, Clemens Hlauschek, Thomas Johansson, Norman Lahr, Alexander Nilsson, and Robin Leander Schröder. Don't reject this: Key-recovery timing attacks due to rejection-sampling in HQC and BIKE. *IACR Trans. Cryptogr. Hardw. Embed. Syst.*, 2022(3):223–263, 2022.

[5] Wen-Ding Li, Ming-Shing Chen, Po-Chun Kuo, Chen-Mou Cheng, and Bo-Yin Yang. Frobenius additive fast fourier transform. In Manuel Kauers, Alexey Ovchinnikov, and Éric Schost, editors, *Proceedings of the 2018 ACM on International Symposium on Symbolic and Algebraic Computation, ISSAC 2018, New York, NY, USA, July 16-19, 2018*, pages 263–270. ACM, 2018.

[6] Carlos Aguilar Melchor, Nicolas Aragon, Slim Bettaieb, Loıc Bidoux, Olivier Blazy, Jean-Christophe Deneuville, Philippe Gaborit, Edoardo Persichetti, Gilles Zémor, and IC Bourges. Hamming Quasi-Cyclic (HQC). `https://pqc-hqc.org/`, 2018.

[7] Joris van der Hoeven and Robin Larrieu. The frobenius FFT. In Michael A. Burr, Chee K. Yap, and Mohab Safey El Din, editors, *Proceedings of the 2017 ACM on International Symposium on Symbolic and Algebraic Computation, ISSAC 2017, Kaiserslautern, Germany, July 25-28, 2017*, pages 437–444. ACM, 2017.